

MCT333/ MCT344: Mechatronic Systems Design

final project report

Name	Code
Marwan hossam El-baz	2200097
Adham Sherif Abdel Nasser	2200445
Ashraf Ahmed	2200403
Ahmed Sayed Ismael	2200410
Ahmed Hossam Eissa	2200138
Mohamed EL Aref	2200438
Nassir Afifi	2200697
Omar Mohamed Hussein Elwy	2200837

Submitted to :

Dr: Mohamed Ibrahim

Dr: Mohamed Nabil

Eng: Youssef Nagaty

Eng: Hossam Moataz

Eng: Mazen talaat

Contents

1)table of figures	4
2)Responsibility sheet :	6
3)Base of car.....	7
3.1)Design of base	7
3.2)base components	10
3.3) base actuator sizing.....	14
According to previous calculations we selected:	15
3.4)base battery sizing.....	16
3.5) PCB of base.....	17
3.6) PID controlling of wheels	21
3.7) Yaw Stabilization System.....	23
4)robotic arm of car.....	26
4.1) overview about arm	26
4.2)operation of robotic arm.....	29
4.3) actuator sizing of arm.....	30
4.4)Battery sizing of arm	34
4.5) PCB of arm.....	35
5) combined car	38
6) communication between MCs	42
6.1)introduction.....	42
6.2)Controller Responsibilities	43
6.3) Serial Communication Protocol.....	46



7)operating sequence	49
7.1) manual operating mode.....	49
7.2)autonomous operating mode	49
8)HMI	51
9) drive link.....	54

1)table of figures

Figure 1 base chassis.....	7
Figure 2 base electronics cover	8
Figure 3 arm holder	9
Figure 4 arm electronics box.....	9
Figure 5 Arduino mega 2560.....	10
Figure 6 L298N H-bridge	10
Figure 7 GA25-370 DC geared motor with integrated encoders.....	11
Figure 8 configuration of mecanum wheels	11
Figure 9 HC-05 Bluetooth module.....	12
Figure 10 rechargeable X-power battery pack	16
Figure 11 PCB schematic.....	17
Figure 12 PCB details	17
Figure 13 robotic arm	27
Figure 14 workspace of arm.....	28
Figure 15 storage of boxes	29
Figure 16 material of robotic arm	30
Figure 17 simulink model of arm.....	31
Figure 18 matlab model of arm.....	31
Figure 19 torque of base servo	32
Figure 20 torque of gripper's servo	32
Figure 21 torque of Second link (elbow) servo	33
Figure 22 torque of First link servo	33
Figure 23 schematic of arm PCB.....	35
Figure 24 PCB arm details	35
Figure 25 Terminal Blocks Part of PCB of arm	36
Figure 26 Pin Header of Servo Driver of PCB arm.....	37
Figure 27 base with it's electronics	38
Figure 28 electronics of robotic arm	38
Figure 29 dimensions of car	39
Figure 30 full car assembled	39
Figure 31 car in real life.....	40
Figure 32 car's poster.....	41
Figure 33 connection between uno,mega and raspherry	42
Figure 34 arduino mega	43



Figure 35 raspberry pi	43
Figure 36 vision of camera and QR scanning	44
Figure 37 camera fixation on arm	44
Figure 38 arduino uno.....	45
Figure 39 Detection of serial interfaces on Raspberry Pi.....	46
Figure 40 : Communication sequence during manual collection mode.	48
Figure 41 autonomous mode workflow	50
Figure 42 dashboard of HMI	51

2) Responsibility sheet :

#	Student name	Student ID	Role	Individual Owned tasks	Shared tasks + contribution %
1	Adham Sherif	2200445	Software Team	- Robot arm analysis & programing (100%)	-Car State machine development (30%) -Autonomous Programming (40%) -Robot Arm wiring (50%)
2	Mohamed El Aref	2200438	Mechanical Team	- Mechanical Cad Design and analysis (Chassis & Robotic Arm) (100%)	- Solving Mechanical Issues (50%) - All robot hardware assembly (30%) - Robot Arm wiring (50%)
3	Ahmed Hossam	2200138	Electrical Team	- Battery and actuator Sizing - IMU PID (100%)	-Base power distribution (50%) -Base movement code (70%) Base PCB design (50%)
4	Ashraf Ahmed	200403	Software Team	- Software Integration (Raspberry pi & Camera) - HMI Design & implementation (100%)	-Car State Machine Development (30%) -Robotic Arm PCB design and layout (50%)

#	Student name	Student ID	Role	Individual Owned tasks	Shared tasks + contribution %
5	Omar Mohamed Hussein Elwy	2200837	Mechanical Team	- Chassis and Robot Manufacturing - Procurement ,Testing and documentation (100%)	- All robot hardware assembly (40%) - Solving Mechanical Issues (50%) - Robotic Arm PCB Manufacturing (50%)
6	Marwan Hossam	2200097	Control Team	- Wheels PID & State machine (100%)	-Car State Machine Development (40%) - Autonomous programming (60%)
7	Ahmed Sayed	2200410	Mechanical Team	- Mechanical cad assembly - Video Production (100%)	- Base electrical schematic (40%) - All robot Hardware Assembly (30%) - Base movement code (30%)
8	Nassir Affi	2200697	Electrical Team	- Base electrical wiring (100%)	- Base electrical schematic (60%) - Base PCB design (50%) - Base power distribution (50%)

3)Base of car

3.1)Design of base

The chassis was designed to provide a strong and stable structure for supporting all mechanical and electrical components of the robot while maintaining a lightweight and compact design as it's made from aluminum . To simplify assembly, maintenance, and future modifications, the chassis was divided into several plates, all manufactured using sheet metal processes including laser cutting and bending.

Base Chassis

The base chassis is the main structural component of the robot. It serves as the foundation that supports the entire system and carries the drive motors, motor mounting brackets, wheels, batteries, electronics, and other mechanical components. Since all loads are transferred through this plate, it was designed to provide sufficient strength and rigidity while keeping the overall weight low.

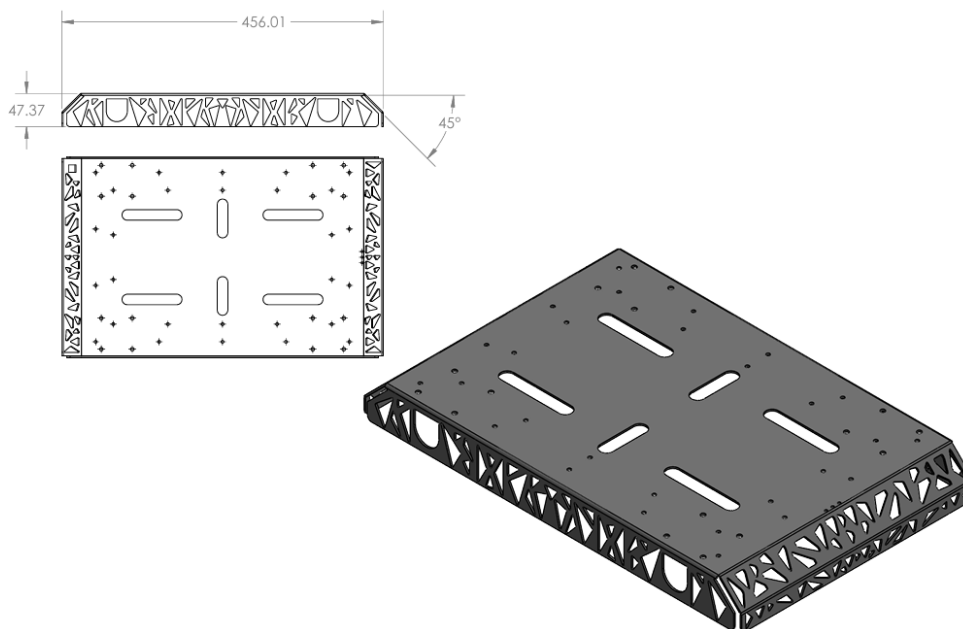


Figure 1 base chassis

Base electronics cover

The electronics cover plate is mounted above the base chassis and is used to protect the electronic components from external damage. It also helps organize the internal layout of the robot and provides a clean and professional appearance. Together, the base chassis and electronics cover plate form the mobile platform of the robot.

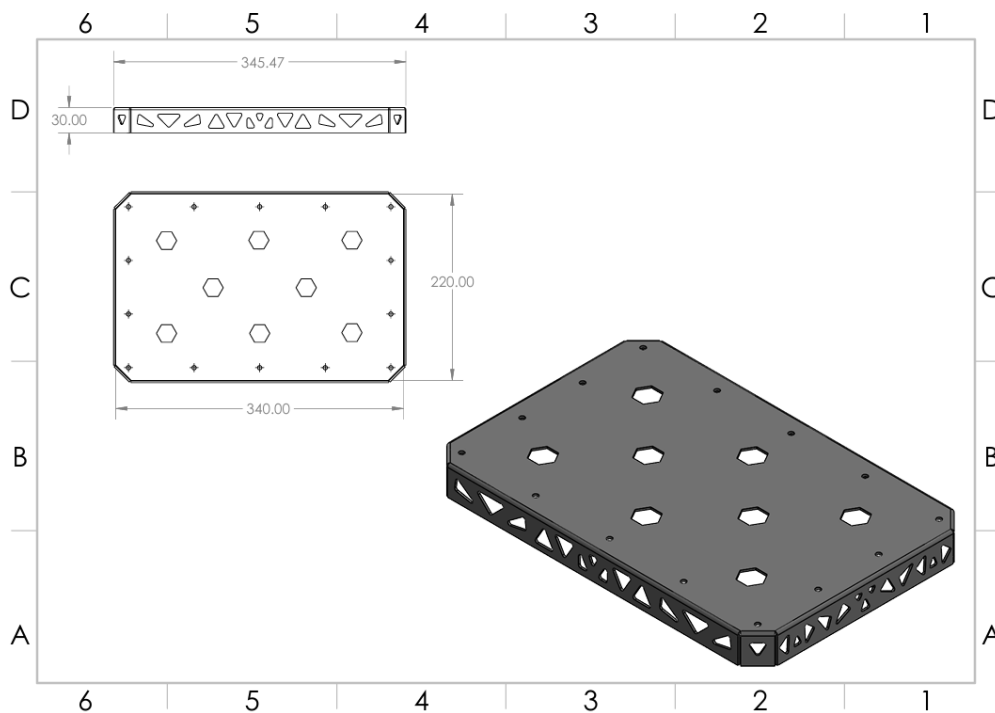


Figure 2 base electronics cover

Arm holder

Arm holder is mounted above base chassis and base electronics box to hold robotic arm ,storage of boxes/arm electronics cover and electrical components of robotic arm .

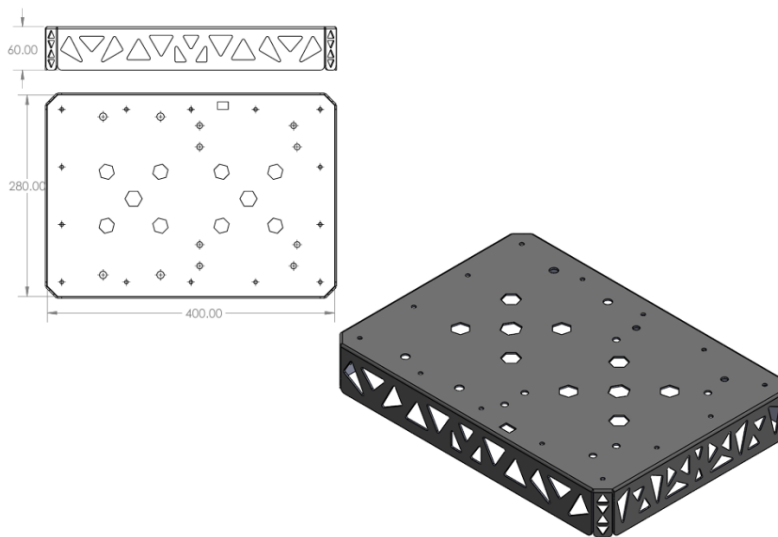


Figure 3 arm holder

Arm electronics cover Plate

The arm mounting plate is designed to support the robotic arm, its electronic components, and the storage box used for holding the collected objects. A key advantage of this design is its modularity. The entire robotic arm assembly can be removed together with its mounting plate from the mobile platform whenever required. This allows the robotic arm and chassis to be handled, maintained, or modified independently without affecting the rest of the system.

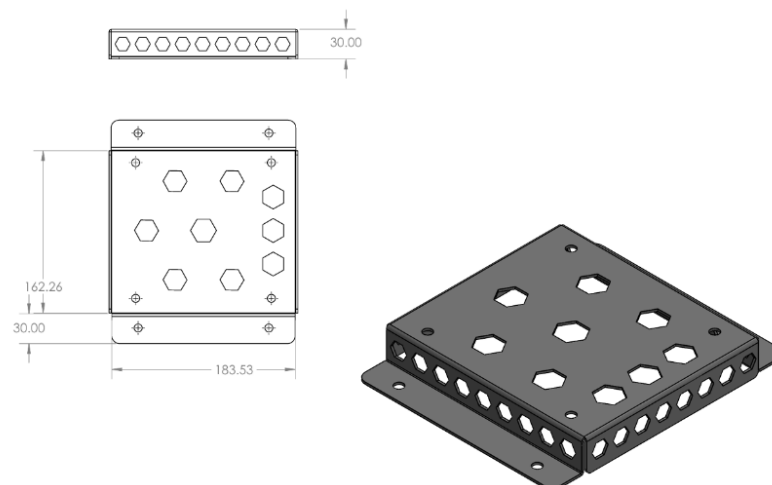


Figure 4 arm electronics box

3.2)base components

1. Arduino Mega 2560 (Main Controller)

The Arduino Mega 2560 serves as the main controller of the mobile base. It receives commands from the Bluetooth module, processes sensor data from the MPU6050, and generates control signals for the motor drivers. Due to its large number of I/O pins and multiple serial communication ports, it is suitable for handling multiple peripherals simultaneously.

Function:

- Central control unit.
- Processes user commands.
- Reads IMU & Encoders data.
- Controls motor speed and direction.



Figure 5 Arduino mega 2560

2. L298N H-Bridge Motor Drivers

Two L298N motor driver modules are used to drive the four DC motors. Each driver controls two motors and allows bidirectional rotation by changing the polarity of the supplied voltage.

Function:

- Amplifies control signals from the Arduino.
- Controls motor direction.
- Provides PWM speed control.

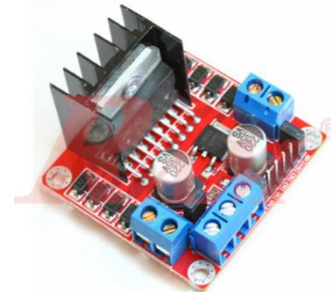


Figure 6 L298N H-bridge

3. DC Geared Motors with Encoders

The mobile base is driven by four **GA25-370 DC geared motors with integrated encoders**. Each motor operates at **12 V**, provides a nominal speed of **250 RPM**, and delivers a maximum torque of approximately **9 kg·cm**. The integrated encoder enables speed measurement and closed-loop control, improving motion accuracy.

Each motor is directly coupled to a Mecanum wheel, allowing independent wheel actuation and enabling omnidirectional robot movement.

Specifications:

- Operating Voltage: 12 V
- Rated Speed: 250 RPM
- Output Torque: 9 kg·cm
- Rated Torque : 4.3 kg·cm
- Quantity: 4 Motors
- Encoder: Integrated Incremental Encoder

Functions:

- Generate the driving torque required for robot motion.
- Enable independent wheel speed control.
- Provide encoder feedback for odometry and motion control.
- Support holonomic movement using the Mecanum wheel configuration.



Figure 7 GA25-370 DC geared motor with integrated encoders

4. Mecanum Wheels (X Configuration)

The mobile base uses four Mecanum wheels arranged in an X-configuration. The angled rollers on each wheel allow the robot to move in any direction without changing its orientation.

Function:

- Forward and backward motion.
- Sideways motion.
- Diagonal motion.
- Rotation about the robot center.

Advantages:

- Full omnidirectional mobility.

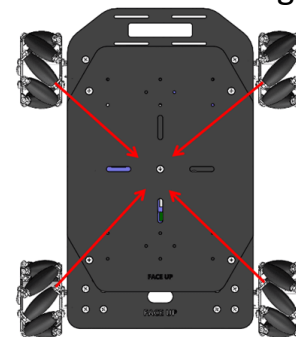


Figure 8 configuration of mecanum wheels

5. HC-05 Bluetooth Module

The HC-05 module provides wireless communication between the robot and a smartphone or remote controller. It enables manual teleoperation during the competition's manual control phase.

Function:

- Wireless command transmission.
- Remote control interface.
- Serial communication with Arduino.



Figure 9 HC-05 Bluetooth module

6. MPU6050 IMU Sensor

The MPU6050 combines a 3-axis accelerometer and a 3-axis gyroscope in a single module. It is used to estimate the robot's orientation and stabilize its heading during movement.

Functions:

- Measure the robot's yaw angle and angular velocity.
- Maintain straight-line motion by correcting heading errors.
- Improve directional stability during movement.
- Enable accurate rotation to a specified target angle.



7. Power ON/OFF Switch

The ON/OFF switch is used as the main power control element for the mobile base. It allows the operator to safely connect or disconnect power from the robot without removing the battery connection. The switch provides a quick method for system startup and shutdown during operation and testing.

Functions:

- Main power control for the robot.
- Safe startup and shutdown of the system.
- Facilitates maintenance and troubleshooting.
- Improves operational safety.



8. Fuse

A fuse is installed in series with the power supply to protect the electrical system from excessive current caused by short circuits or component failures. If the current exceeds the fuse rating, the fuse disconnects the circuit, preventing damage to the battery, controllers, motors, and wiring.

Functions:

- Overcurrent protection.
- Short-circuit protection.
- Prevents damage to electronic components.
- Enhances overall system safety.



3.3) base actuator sizing

Total mass of the car = Mass of chassis + Mass of robot arm = 5500+700= 6200 gm = 6.2 kg

Frontal area = 0.011 m²

Take coefficient of rolling resistance = 0.03

MATLAB code :

```
clc;
clear;

%% =====
%   INPUT PARAMETERS
%% =====

m = 6.2;           % Total mass (kg)
Rw = 0.03;         % Wheel radius (m)
V = 0.5;           % Desired velocity (m/s)
t_acc = 0.1;        % Acceleration time (s)
mu_rr = 0.03;       % Rolling resistance coefficient
g = 9.81;           % Gravity (m/s^2)
N_motors = 4;       % Number of motors
Cd = 1.1;           % Drag coefficient (optional)
rho = 1.2;          % Air density (kg/m^3)
Af = 0.011;         % Frontal area (m^2) (approx)

%% =====
%   CALCULATIONS
%% =====

% Linear acceleration
a = V / t_acc;

% Rolling resistance force
F_roll = mu_rr * m * g;

% Acceleration force
F_acc = m * a;

% Air drag force (can be neglected at low speed)
F_air = 0.5 * Cd * rho * Af * V^2;

% Total force required
```

```
F_total = F_roll + F_acc + F_air;  
  
% Total torque required for robot  
T_total = F_total * Rw;  
  
% Torque per motor  
T_motor = T_total / N_motors;  
  
% Add safety factor (x2)  
SF = 2;  
T_design = T_motor * SF;  
  
% Convert to kg.cm  
T_kgcm = T_design * 10.2;  
  
%% =====  
%   RESULTS  
%% =====  
  
fprintf('Total Required Torque (Robot) = %.4f N.m\n', T_total);  
fprintf('Torque per Motor (No SF) = %.4f N.m\n', T_motor);  
fprintf('Design Torque per Motor (With SF) = %.4f N.m\n', T_design);  
fprintf('Design Torque per Motor = %.3f kg.cm\n', T_kgcm);
```

Results :

```
Total Required Torque (Robot) = 0.9848 N.m  
Torque per Motor (No SF) = 0.2462 N.m  
Design Torque per Motor (With SF) = 0.4924 N.m  
Design Torque per Motor = 5.022 kg.cm
```

Selected Motor :

According to previous calculations we selected:

GA25-370 MOTOR ENCODER 12V 250RPM 9KG.CM

3.4)base battery sizing

1) Omni (4 Dc motors + Arduino mega + Bluetooth module+ IMU)

- $I_{\text{motors}} = 4 \times 1.5 = 6 \text{ A}$ (12v)
 - $I_{\text{arduino+sensors}} = 1 \text{ A}$ (5v)
 - Total power = $6 \times 12 + 5 \times 1 = 77 \text{ watt}$
 - $I_{\text{battery}} = 77/12 = 6.4 \text{ A}$
- We will use Rechargeable X – Power Battery Pack (11.3V to 12V) 4000mAh With BMS 20A
- Operating time = $(60 \times 4)/6.4 = 37.5 \text{ minutes}$



Figure 10 rechargeable X-power battery pack

3.5) PCB of base

1 Schematics

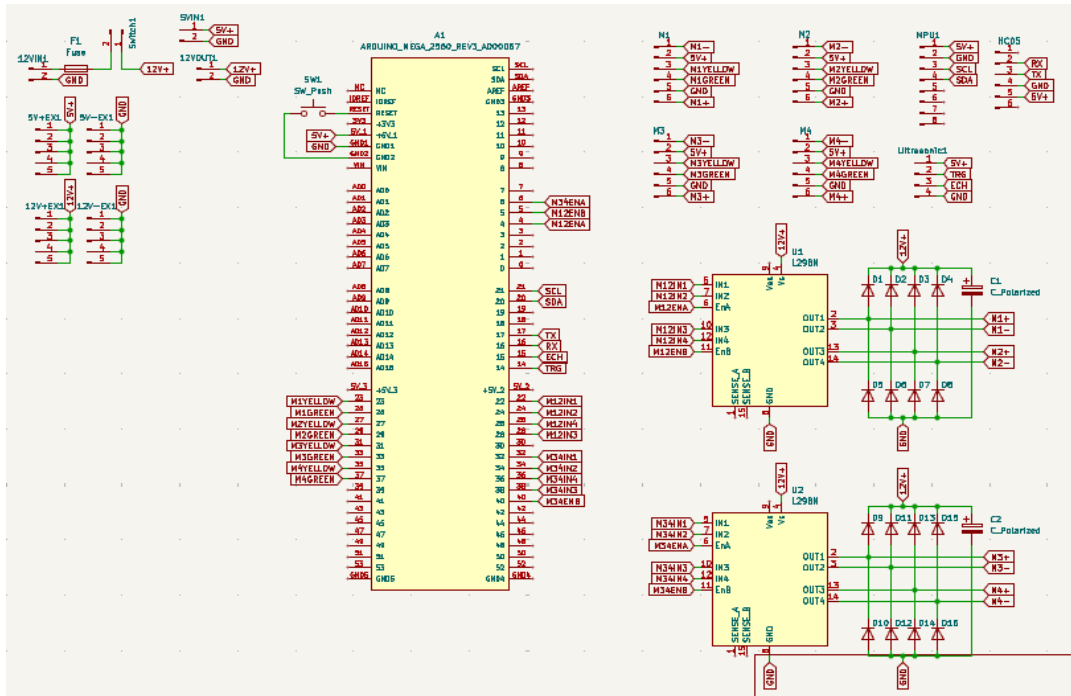


Figure 11 PCB schematic

2. PCB Details

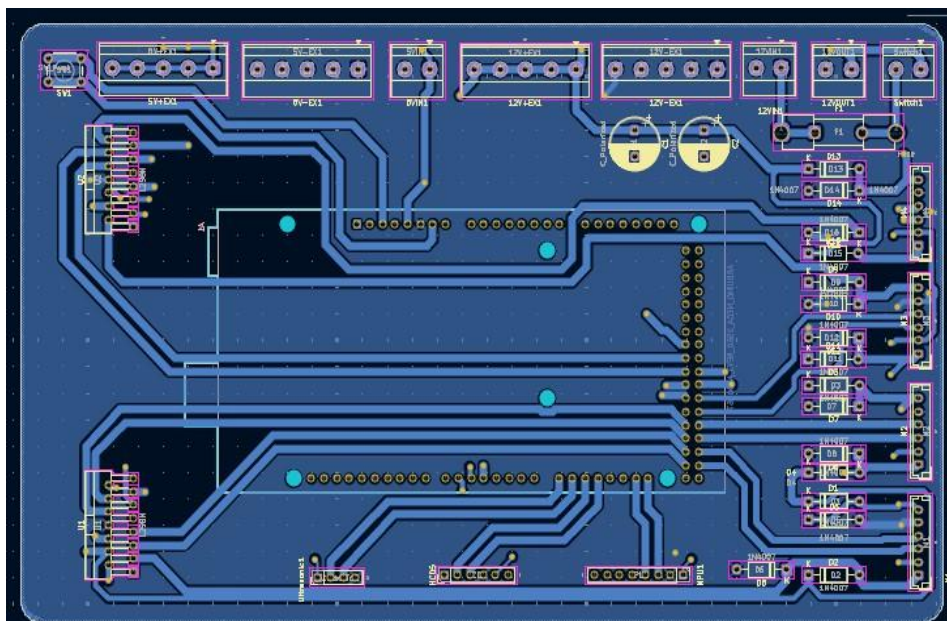
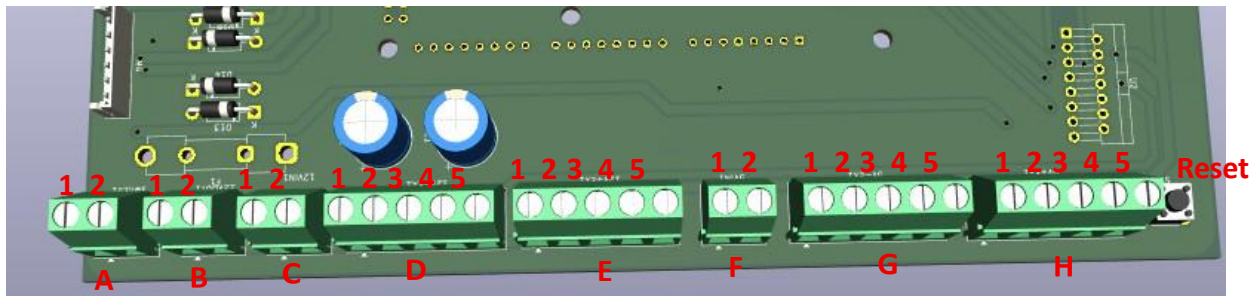


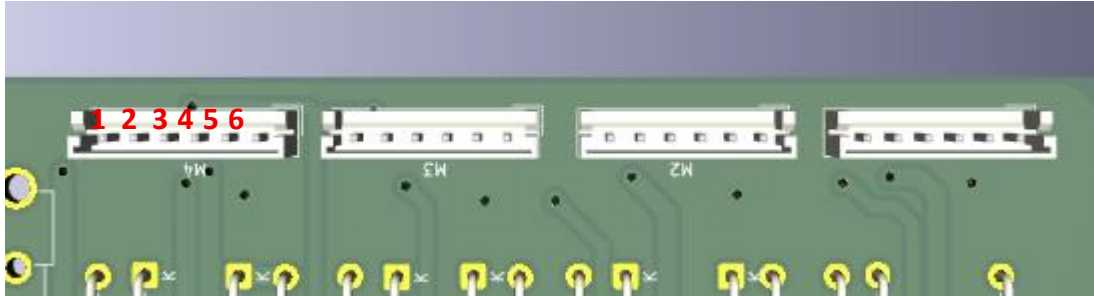
Figure 12 PCB details

A. Power Part



- A: Switch Terminal Block 1: Switch Pin 1
- 2: Switch Pin 2
- B: 12V Output for other board or modules 1: 12V+ Pin
- 2: GND Of 12V
- C: 12V Input for the board 1: 12V+ Input
- 2: GND Input of 12V
- D: Negative 12V "GND" Extension Terminal 1, 2, 3, 4 and 5: Additional 12V GND output
- E: Positive 12V+ Extension Terminal
- 1, 2, 3, 4 and 5: Additional 12V+ positive output
- F: 5V input from stepdown 1: 5V+ Input
- 2: GND or Negative pin of Stepdown
- G: 5V Negative "GND" Additional Terminal
- 1, 2, 3, 4 and 5: Additional 5- Negative output
- H: 5V+ Positive Additional Terminal
- 1, 2, 3, 4 and 5: Additional 5+ positive output
- Reset: Reset button of Arduino

B. Motors Part



4 Motor Connectors:

1: Motor Positive pin 2: GND

3: Green Jumper connected to arduino 4: Yellow Jumper connected to Arduino 5: VCC "5V+"

6: Motor Negative Pin

Each Connector have the same Pins "They are based on how the Encoder jumper will be arranged"

C. Sensors Part



A: Ultrasonic Sensor 1: VCC "5V"

2: Trigger

3: Echo

4: GND

B: Bluetooth Module 1: Unconnected

2: RX "Connected to TX On Arduino" 3: TX "Connected to RX On Arduino" 4: GND

5: VCC "5V+"

6: Unconnected

C: IMU

1: Unconnected

2: Unconnected

3: Unconnected

4: Unconnected

5: SDA

6: SCL

7: GND

8: +5V VCC

3.6) PID controlling of wheels

1. Encoder Resolution (CPR) and Rotational Speed (RPM) Calculation

At the onset of developing the robot's locomotion system, understanding the exact characteristics of the utilized motors and encoders was crucial. We determined the number of pulses generated by the encoder per full wheel revolution (CPR - Counts Per Revolution) to be exactly **308 pulses/rev**. Using this parameter, an algorithm was implemented within the microcontroller to calculate the actual wheel speed in Revolutions Per Minute (RPM). This is achieved by measuring the change in pulse count (Delta Ticks) over a fixed time interval (Interval = 50 ms), utilizing the following mathematical relation: $RPM = (\Delta \text{Ticks} \times 60000) / (CPR \times \text{Interval})$

2. Application of PID Controlling

Given that the robotic platform utilizes Mecanum wheels and weighs approximately 6.2 kg, the accuracy of its kinematics heavily relies on the four wheels rotating at identical speeds when commanded. Any slight discrepancy in wheel speed—due to varying friction or uneven weight distribution—results in drifting and the failure of straight, lateral, or diagonal trajectories. To mitigate this, a closed-loop control system (PID Controller) was programmed for each wheel independently using manual tuning. The system reads the actual speed from the encoder, compares it to the Target RPM, and adjusts the motor's power output (PWM) to unify the speeds.

3. Initial Tuning Trials

To achieve the optimal response, we began by assigning various parameters to the PID controller to observe the motors' behavior. The testing phases included three main configurations:

- **Trial 1 ($K_p = 1.2$, $K_i = 0.4$, $K_d = 0.8$):**

Observation: The motors took too long to reach the target speed and struggled to overcome the inertia of the heavy chassis. Furthermore, the relatively high K_d value amplified encoder noise

- **Trial 2 ($K_p = 5.0$, $K_i = 0.0$, $K_d = 0.0$):**

Observation: By relying entirely on proportional control and disabling the integral term, the initial acceleration was sufficiently aggressive. However, the motors suffered from a significant steady-state error. Because of the heavy load, the proportional gain alone could not push the motors to the exact target RPM, leaving a

constant performance gap.

4. Final Adopted Values and Engineering Analysis

Following the iterative manual tuning process, the optimal parameters for the Mecanum wheels were finalized as follows:

- $K_p = 3.0$
- $K_i = 5.0$
- $K_d = 0$

Engineering Justification for the Final Parameters:

- **Why ($K_p = 3.0$)?** This proportional gain provides a sufficiently strong initial "kick" to break the static friction and chassis inertia immediately upon receiving movement commands. It is balanced perfectly to drive the motor swiftly toward the target without causing aggressive overshoot.
- **Why a relatively high ($K_i = 5.0$)?** In velocity control systems, the integral term plays the most critical role. A high value of 5.0 quickly accumulates and compensates for any speed deficits caused by the robot's weight or floor friction. This guarantees the complete elimination of steady-state error within fractions of a second, forcing all four wheels to strictly match the requested RPM regardless of individual mechanical variances.
- **Why zero derivative gain ($K_d = 0.0$)?** This was a deliberate engineering decision. Derivative control reacts to the rate of error change. Since velocity is calculated from encoder ticks every 50 ms, the readings inherently contain slight quantization noise. Introducing K_d would exponentially amplify this noise, sending erratic PWM signals and causing the motors to vibrate or "buzz." For motor speed control, a well-tuned **PI controller** is generally ideal and highly stable compared to a full PID.

3.7) Yaw Stabilization System

The MPU6050 is an Inertial Measurement Unit (IMU) that integrates a 3-axis accelerometer and a 3-axis gyroscope. In the proposed mobile robot, the sensor is used primarily for orientation estimation and heading stabilization. The measured yaw angle is continuously monitored and used as feedback for a PID controller to maintain the desired robot heading during autonomous navigation.

Sensor Initialization

The MPU6050 communicates with the Arduino Mega through the I2C protocol. During system startup, the sensor is initialized and its Digital Motion Processor (DMP) is enabled. The DMP internally processes the raw accelerometer and gyroscope measurements and provides filtered orientation data, reducing computational load on the microcontroller and improving measurement stability.

Calibration Process

Before operation, the robot remains stationary for a short calibration period. During this period, the initial yaw angle is measured and stored as a reference offset.

The measured offset is later subtracted from all subsequent yaw readings, ensuring that the robot starts with a heading of approximately 0° regardless of its physical orientation at startup.

The yaw angle used by the controller is calculated as:

$$\text{Current Yaw} = \text{Measured Yaw} - \text{Yaw Offset}$$

Orientation Measurement

During autonomous operation, the Arduino continuously reads orientation data from the MPU6050 FIFO buffer.

The DMP generates quaternion data, which is then converted into Euler angles:

- Yaw (rotation about the vertical axis)
- Pitch (rotation about the lateral axis)
- Roll (rotation about the longitudinal axis)

For this project, only the yaw angle is used for navigation and heading control.

Yaw PID Controller

To maintain the desired heading, a PID controller is implemented.

The controller continuously compares:

$$\text{Yaw Error} = \text{Yaw Setpoint} - \text{Current Yaw}$$

The resulting error is processed by the PID controller according to:

$$\text{PID Output} = K_p \cdot \text{Error} + K_i \cdot \int \text{Error} \, dt + K_d \cdot d(\text{Error})/dt$$

The PID gains used in the implementation are:

- $K_p = 4.0$
- $K_i = 0.05$
- $K_d = 0.5$

The controller output represents a corrective rotational command that compensates for heading deviations.

Deadband Implementation

To prevent unnecessary corrections caused by sensor noise and small disturbances, a deadband is implemented.

During autonomous operation:

- If the heading error is less than $\pm 1^\circ$, no correction is applied.
- If the heading error exceeds $\pm 1^\circ$, the PID controller generates a corrective action.

This approach improves system stability and prevents oscillatory behavior.

Integration with Mecanum Wheel Control

The PID correction signal is combined with the robot rotational velocity command before being applied to the Mecanum wheel kinematics.

As a result, whenever the robot deviates from its desired heading due to wheel slip, uneven surfaces, or external disturbances, the controller automatically adjusts the wheel speeds to restore the correct orientation.

This enables:

- Accurate straight-line motion.
- Improved lateral (sideways) movement.
- Stable diagonal motion.
- Precise rotational maneuvers.
- Enhanced autonomous navigation performance.



Functional Role in the Project

The MPU6050 serves as the primary heading feedback sensor for the mobile base. Its measurements allow the robot to continuously estimate its orientation and maintain a desired heading through closed-loop PID control. This significantly improves navigation accuracy and repeatability during autonomous operation.

4)robotic arm of car

4.1) overview about arm

1 Robot Description

The robotic arm is responsible for grasping and handling objects during the robot's operation. It was designed as a five-degree-of-freedom (5-DOF) manipulator to provide the flexibility required to reach, pick, and place objects accurately.

The robotic arm consists of the following joints and components:

- * Rotating Base
- * Shoulder Joint
- * Elbow Joint

- * Wrist Joint
- * Gripper

The rotating base allows the arm to rotate around the vertical axis, increasing the working area of the robot. To support the loads acting on this joint, a bearing was incorporated between the rotating and fixed bases. The bearing carries the applied loads and reduces the forces transmitted to the motor, ensuring smoother motion, better stability, and longer motor life.

The remaining joints, including the shoulder, elbow, and wrist, provide the required movement for positioning the gripper. The gripper is the end-effector of the arm and is responsible for grasping and holding the target objects securely.

The robotic arm components were manufactured using 3D printing technology. Plastic material was selected because it provides a good balance between strength, low weight, ease of manufacturing, and cost. The use of 3D printing also allowed complex shapes to be produced quickly and accurately while reducing manufacturing time and material waste.

2 Robot Configuration

- Robot Type: Articulated Manipulator
- Degrees of Freedom (DOF): 5
- Joint Configuration: R-R-R-R
- End Effector: Two-finger Gripper
- Actuation: Servo Motors
- Application: Pick-and-Place Operations

3 Robot Specifications

Parameter Value

Number of Joints 4

Joint Type Revolute

Degrees of Freedom 5

End Effector Gripper

Total Weight 695 gm

Motor Type Servo Motors

Control Method Joint-Space Position Control

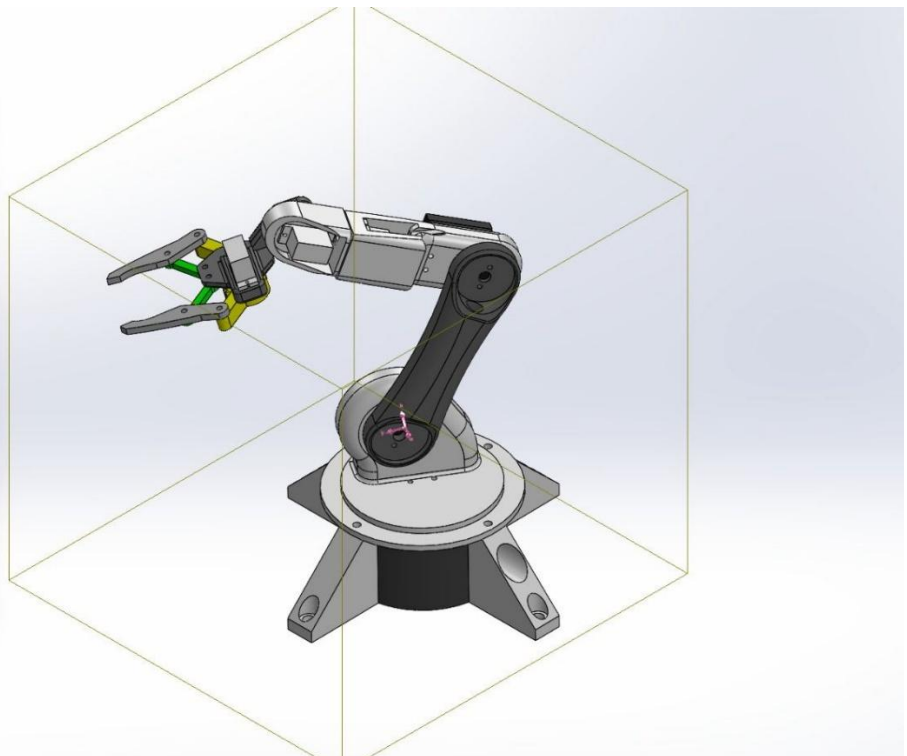
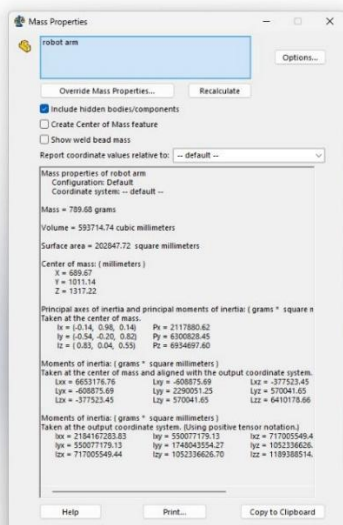


Figure 13 robotic arm

4 Link Dimensions

Link Length (m)

Link 1 0.0564

Link 2 0.17

Link 3 0.15

Link 4 0.08364

The exact dimensions are obtained from the CAD model and are used in the kinematic and dynamic analysis.

Units in the following figure are in meters

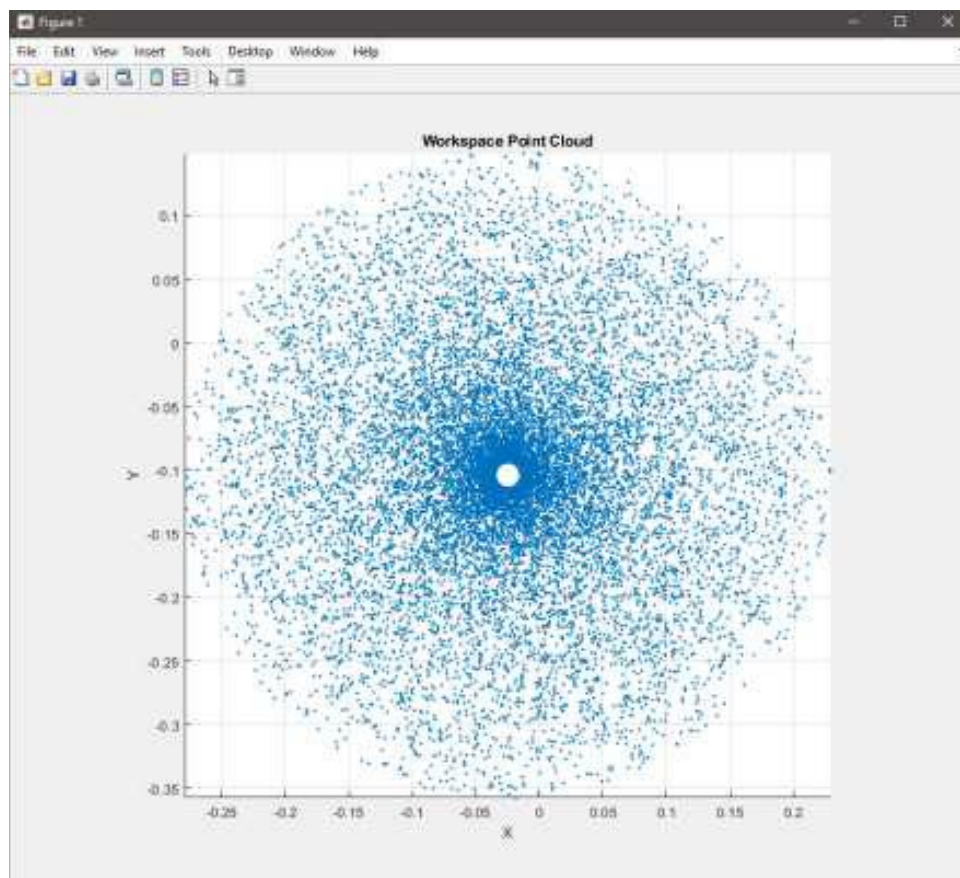


Figure 14 workspace of arm

4.2)operation of robotic arm

To determine whether the robot arm could successfully reach and pick up a box, the geometry of the system was analyzed. The height of the box stand was 40 cm, while the horizontal distance between the robot and the stand was 20 cm. Using these dimensions, the required reach distance was calculated and compared with the maximum reach of the robot arm. This analysis was used to verify that the target box location was inside the robot arm workspace and could be reached safely before executing any motion.

After completing the calculations, a software algorithm was developed to determine whether the robot arm could successfully reach and pick up the target object. The code evaluates the target position and verifies its reachability before executing the pick-and-place sequence.

The robot arm is controlled using an Arduino Uno connected to a PCA9685 servo driver. The system controls four servo motors corresponding to the base, shoulder, elbow, and gripper joints. Smooth motion algorithms were implemented to move the servos gradually between positions, reducing vibration and improving positioning accuracy.

To save boxes in storage which is mounted on car easily we have made a special design to make sure boxes won't fall out it's position

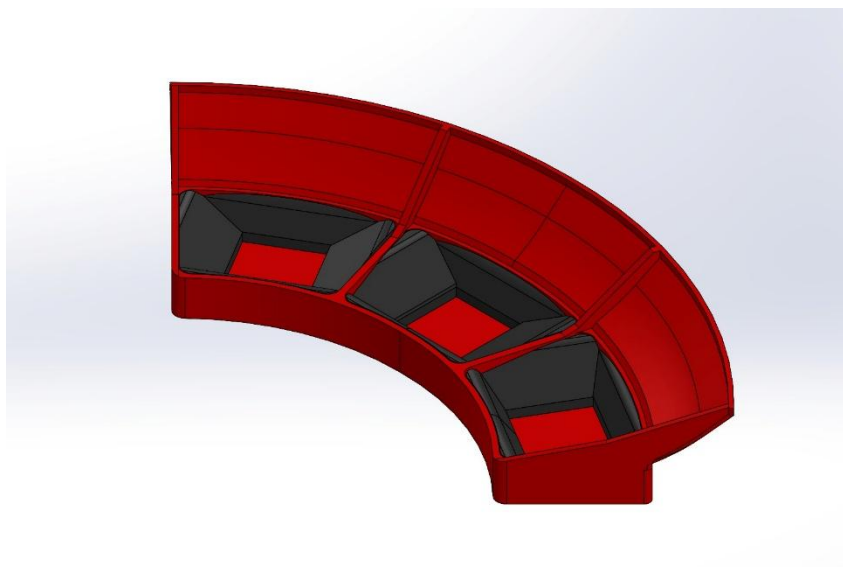


Figure 15 storage of boxes

4.3) actuator sizing of arm

- Using robot arm material PET Plastic (Polyethylene Terephthalate)

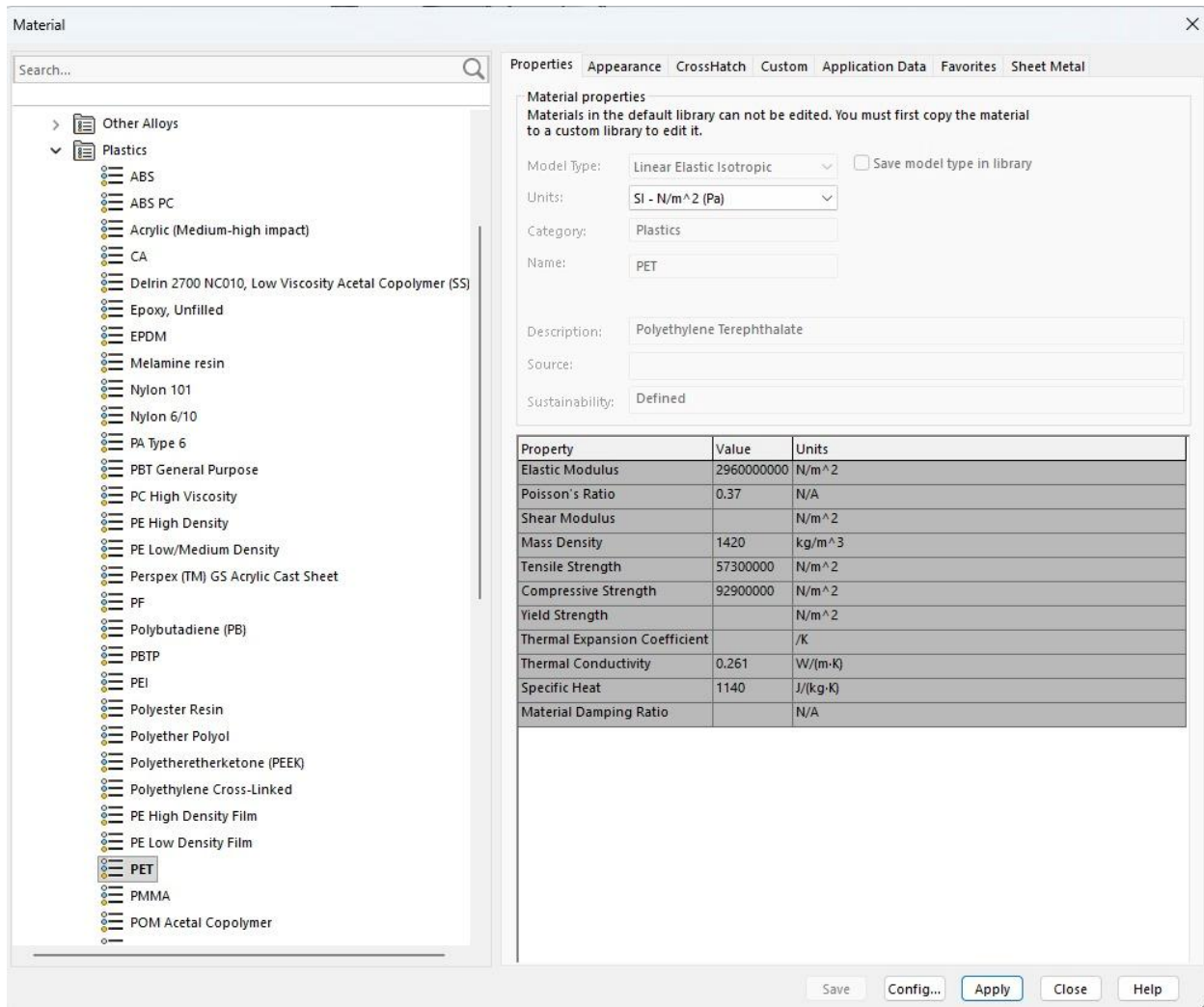


Figure 16 material of robotic arm

Simulink and MATLAB Models

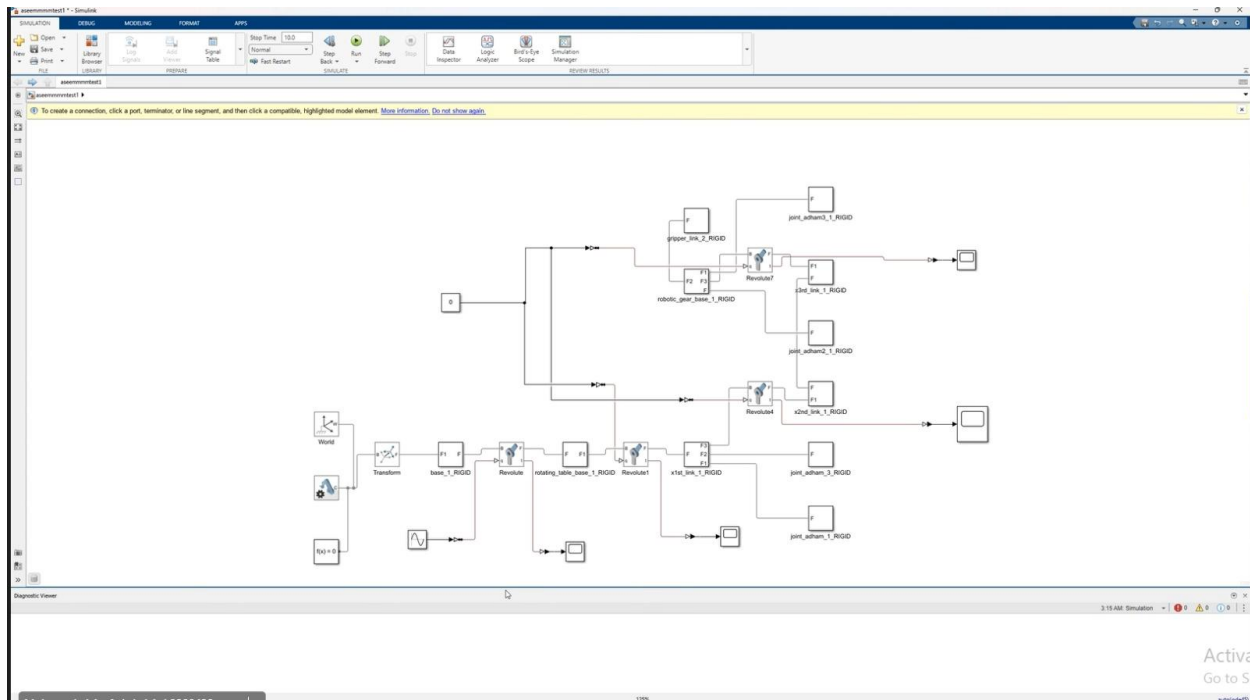


Figure 17 simulink model of arm

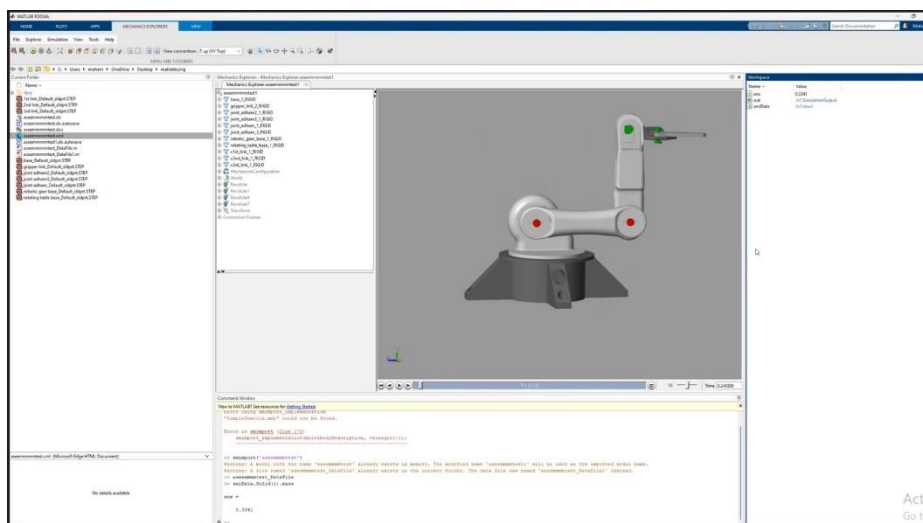


Figure 18 matlab model of arm

1) torque of Base servo

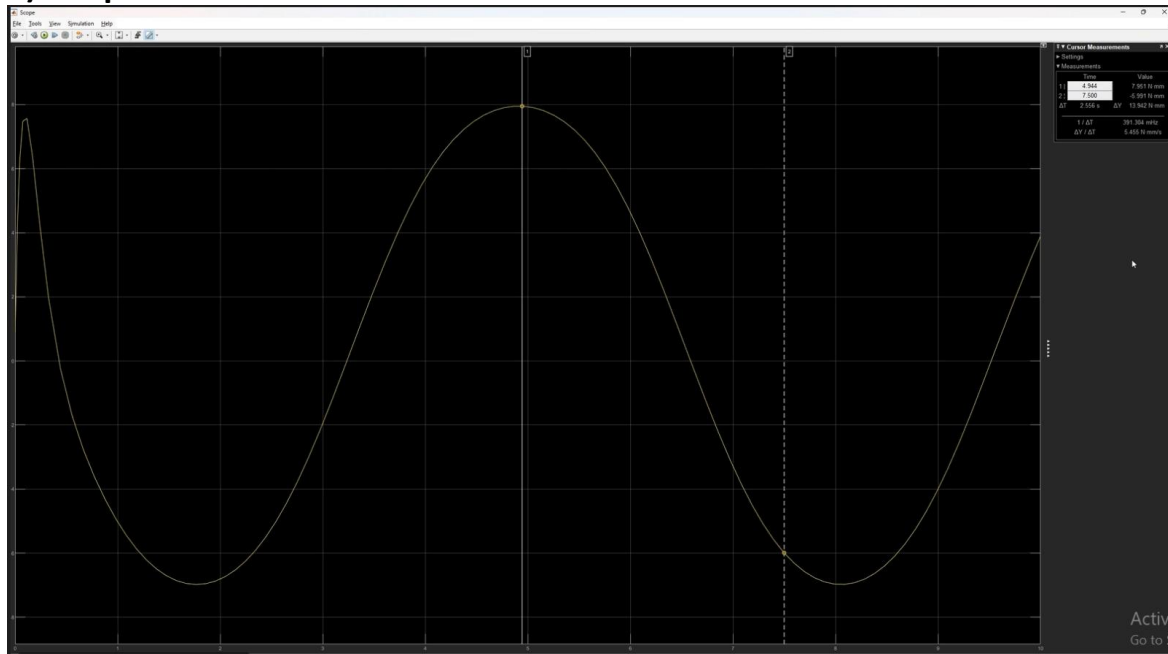


Figure 19 torque of base servo

- Torque = 8 N.mm

2) torque of Gripper servo vertical

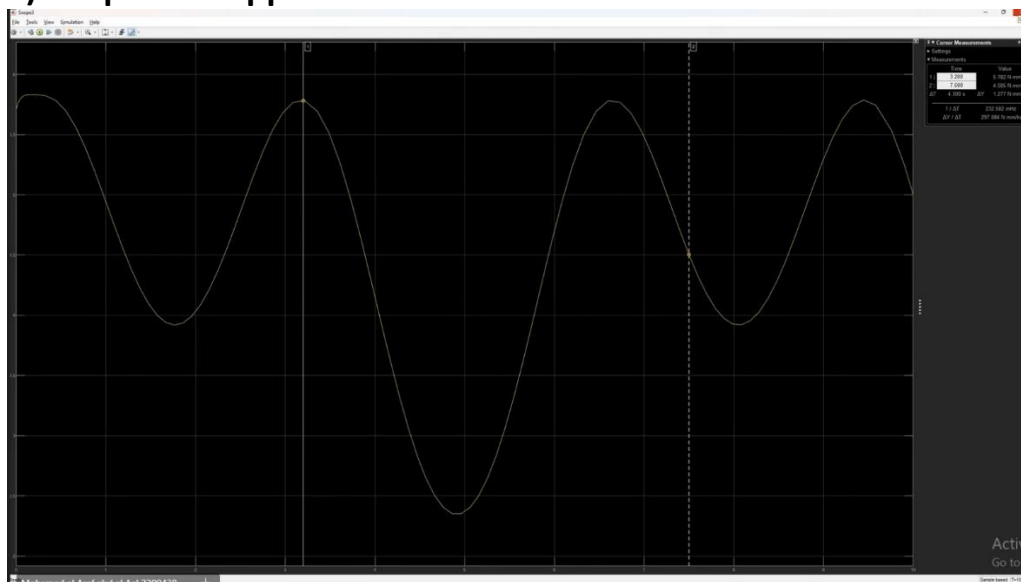


Figure 20 torque of gripper's servo

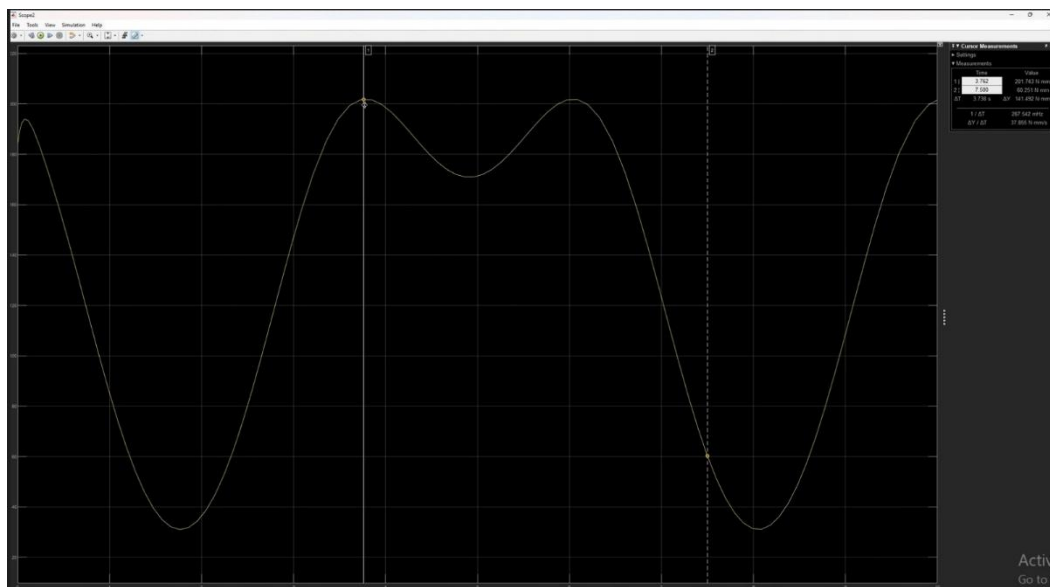
Torque = 6 N.mm

The screenshot displays a software window titled "Report" with a menu bar (File, Tools, View, Simulation, Help) and a toolbar. The main area features a grid with a blue sine wave. A vertical dashed line is positioned at the peak of the wave, with a yellow dot at the peak and a small crosshair. To the right, a panel titled "1.4 Curve Measurements" contains a "Settings" section and a "Measurements" table.

Curve	Value
1	4.931
2	1.000
ΔT	2.569 s
ΔY	110.336 N/mm
$1/\Delta T$	389.285 1/s
$\Delta Y/\Delta T$	48.808 N/mm/s

At the bottom left, a status bar shows "Mohamed al Arab shot of Asl 2200438". At the bottom right, there is a watermark "Active Go to" and a small text "Sample Asset 1/1/18".

4) torque of First link servo:



33 | Page

34 | Page

4.5) PCB of arm

1.Schematic

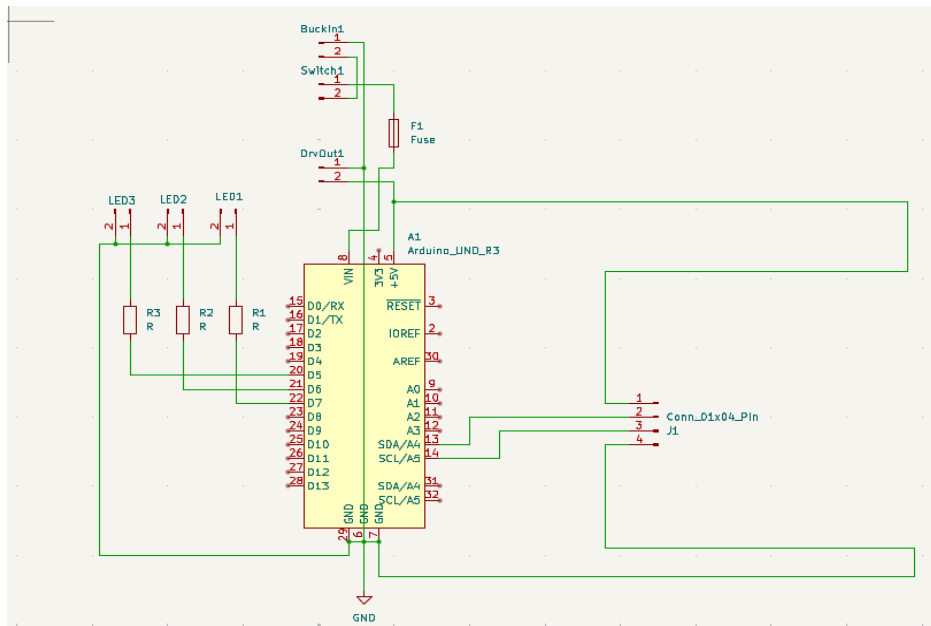


Figure 23 schematic of arm PCB

2. PCB Details

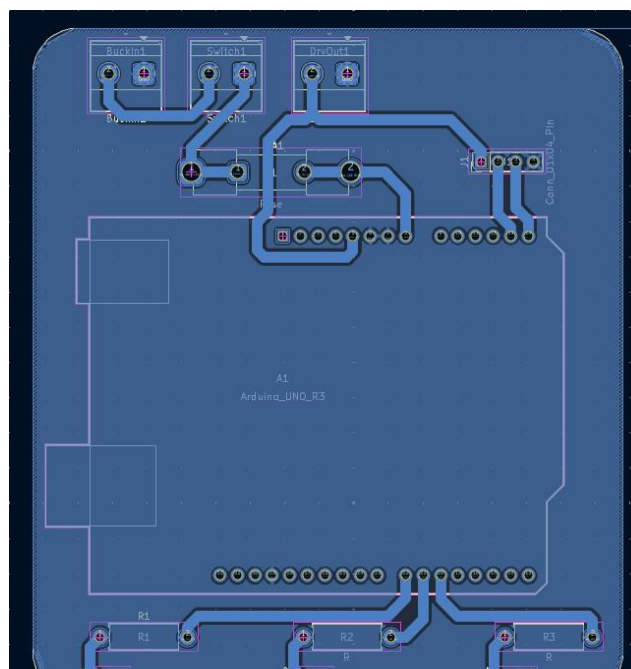


Figure 24 PCB arm details

3: Terminal Blocks Part

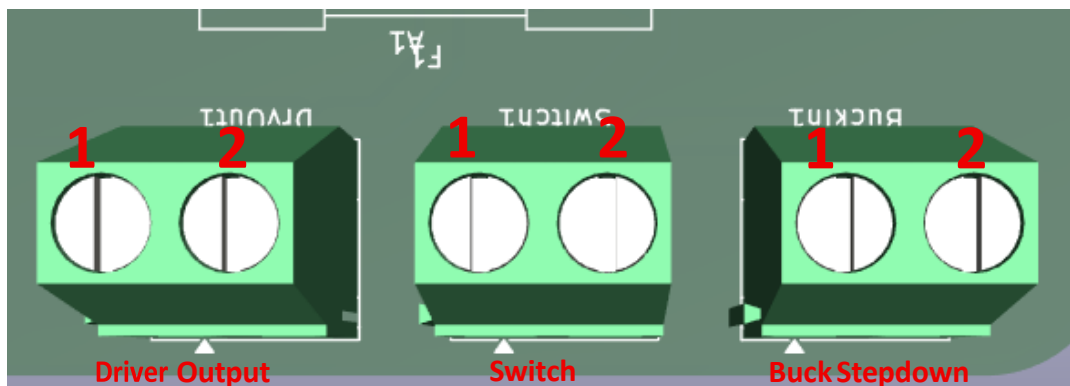


Figure 25 Terminal Blocks Part of PCB of arm

A: Driver Output 1: GND
2: VCC "5V+"

B: Switch
1: Switch pin 1
2: Switch Pin 2

C: Buck Stepdown 1: GND "5V-"
2: VCC "5V+"

4: Pin Header of Servo Driver

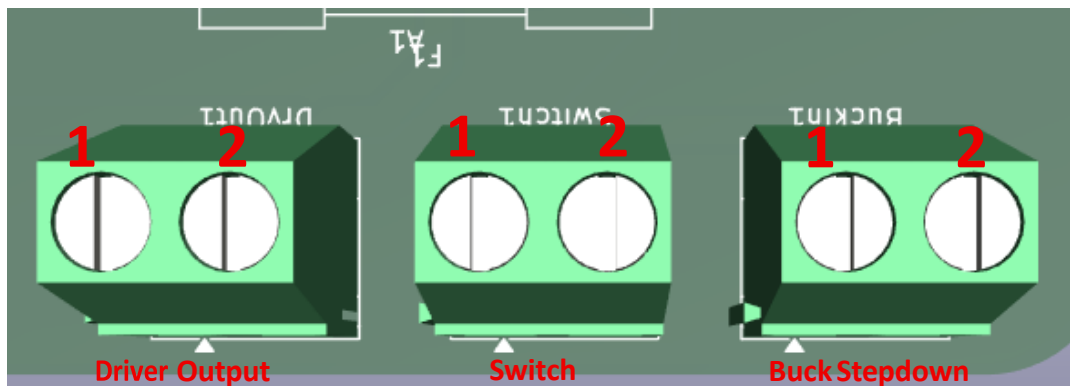


Figure 26 Pin Header of Servo Driver of PCB arm

- 1: GND
- 2: Arduino Pin
- 3: Arduino Pin
- 4: VCC "5V+"

C: LEDs Part

- 1: GND
- 2: Positive pin of LED

5) combined car

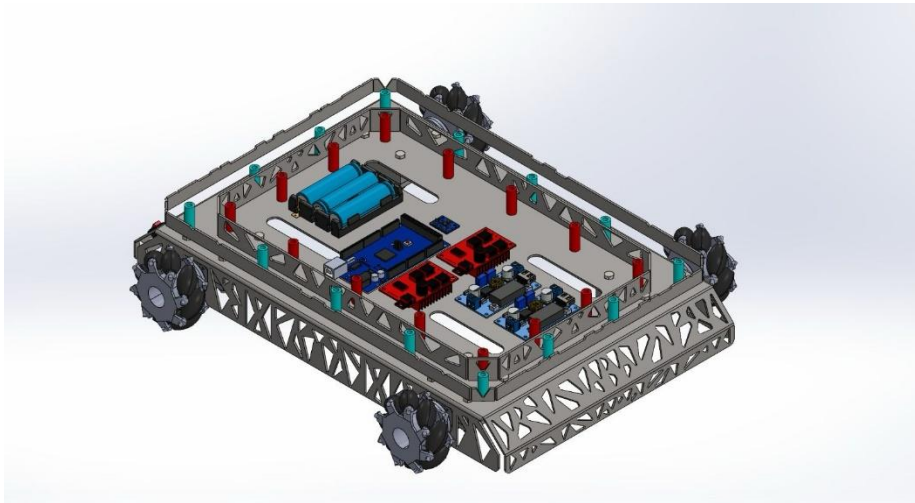


Figure 27 base with it's electronics

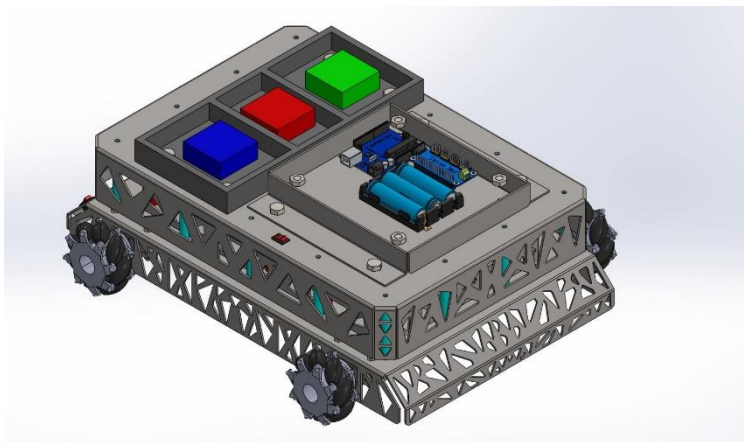


Figure 28 electronics of robotic arm

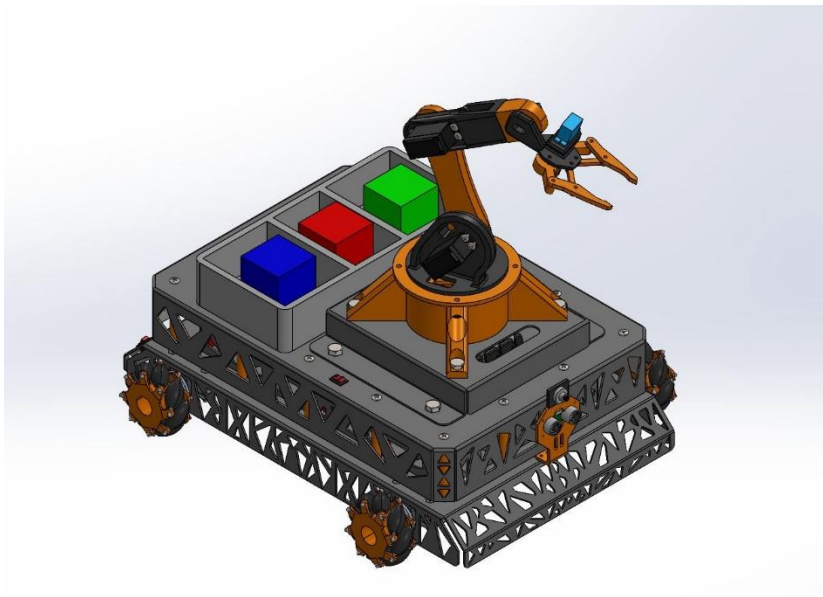


Figure 30 full car assembled

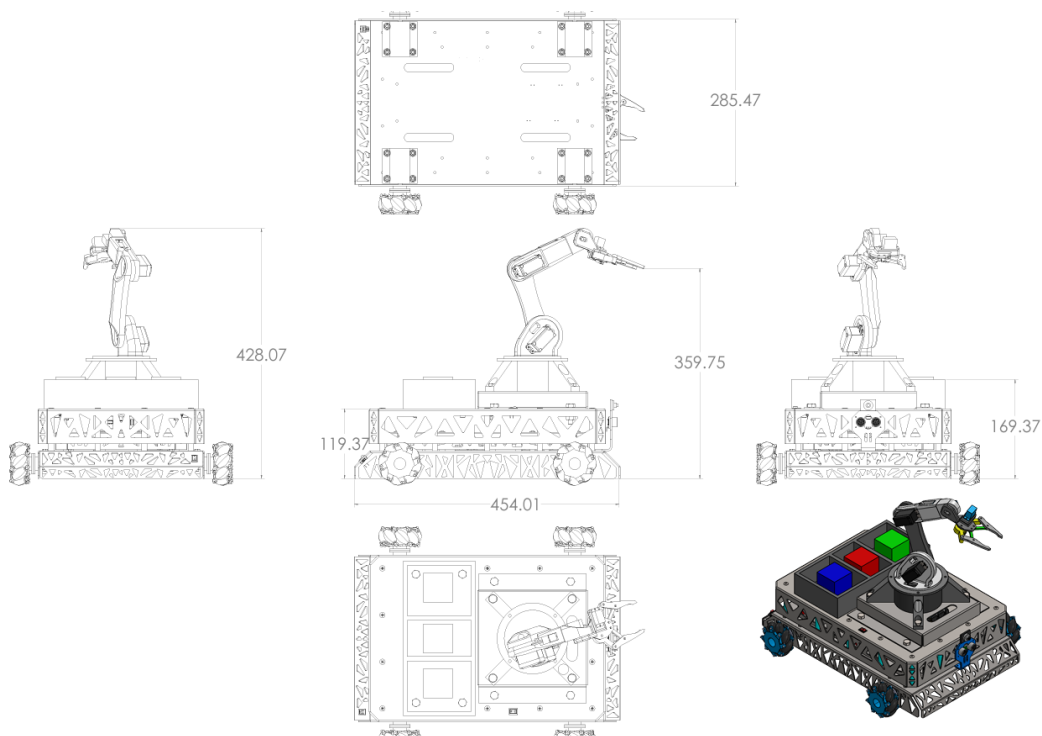


Figure 29 dimensions of car

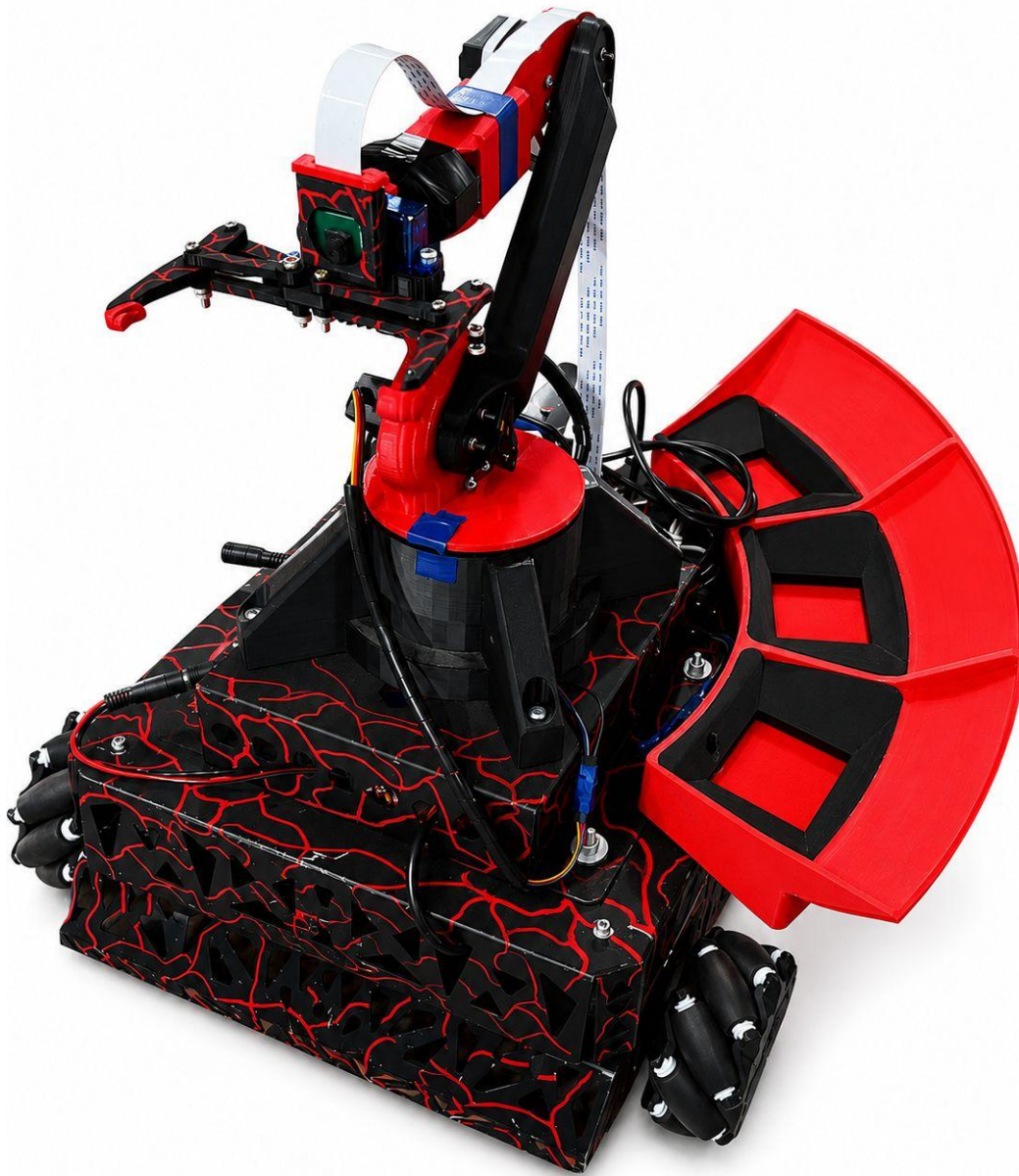


Figure 31 car in real life



Figure 32 car's poster

6) communication between MCs

6.1) introduction

The proposed robot employs a distributed communication architecture composed of three controllers: Arduino Mega, Raspberry Pi 4, and Arduino Uno. The system was designed using a multi-controller approach to distribute computational tasks and improve system performance and reliability

Instead of assigning all system functions to a single controller, the project adopts a distributed architecture in which each controller performs a dedicated set of tasks. This design reduces computational load, enables parallel processing, simplifies debugging, and improves system modularity and maintainability.

Communication between system components is achieved through **USB serial communication**. The Raspberry Pi acts as the communication hub between the motion controller (Arduino Mega) and the manipulator controller (Arduino Uno). This architecture enables data exchange, synchronization of tasks, and real-time monitoring

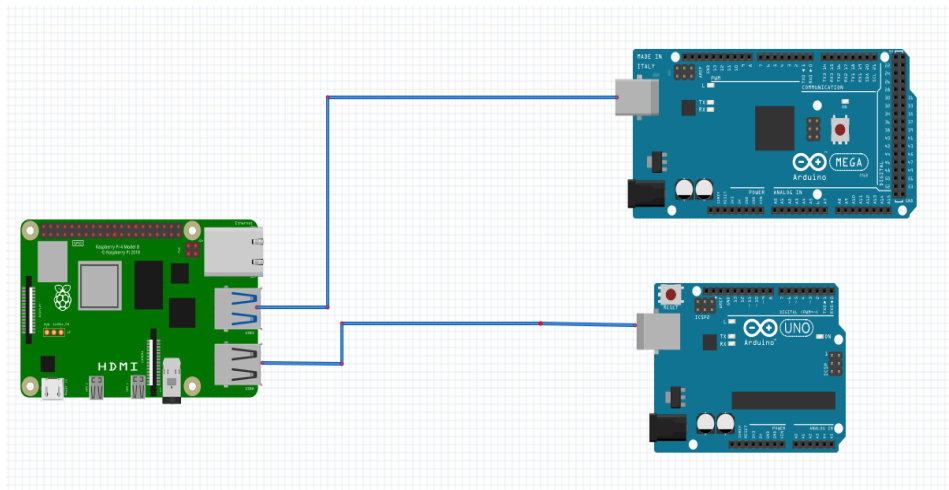


Figure 33 connection between uno, mega and raspberry

6.2)Controller Responsibilities

Arduino Mega Responsibilities :

Arduino Mega serves as the low-level motion controller of the robot. It is responsible for all real-time motion-related tasks and autonomous navigation functions.

Reading wheel encoder signals.



Figure 34 arduino mega

Calculating wheel RPM values.

Acquiring IMU measurements.

Estimating robot position using odometry.

Executing PID speed control algorithms.

Generating PWM signals for DC motors.

Managing autonomous navigation sequences.

Communicating with Raspberry Pi.

Raspberry Pi Responsibilities :

Raspberry Pi 4 acts as the high-level processor and communication coordinator of the proposed system.



Figure 35 raspberry pi

capturing images from the Raspberry Pi Camera.
Performing QR code recognition.
Managing serial communication.
Coordinating task execution between controllers.
Processing telemetry data.
Updating the Human-Machine Interface.
Monitoring robot status in real time.



Figure 36 vision of camera and QR scanning

Raspberry Pi was selected because computer vision and graphical user interface operations require significantly higher computational resources than those available on conventional microcontrollers.



Figure 37 camera fixation on arm

Arduino Uno Responsibilities :

Arduino Uno serves as the dedicated manipulator controller responsible for robotic arm operations.

Receiving commands from Raspberry Pi.

Controlling servo motors through PCA9685.

Executing pick sequences.

Executing place sequences.

Controlling visual LED indications.

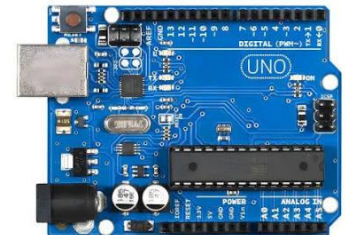


Figure 38 arduino uno

The proposed system follows the control architecture. Raspberry Pi performs high-level decision making and computationally intensive operations, while Arduino Mega and Arduino Uno perform low-level real-time hardware control tasks.

Controller	Classification
Raspberry Pi 4	High-Level Controller
Arduino Mega	Low-Level Motion Controller
Arduino Uno	Low-Level Manipulator Controller

6.3) Serial Communication Protocol

Two independent serial communication channels were established in the proposed system.

Communication Link	Interface	Baud Rate
Raspberry Pi ↔ Arduino Mega	USB Serial	115200 bps
Raspberry Pi ↔ Arduino Uno	USB Serial	9600 bps

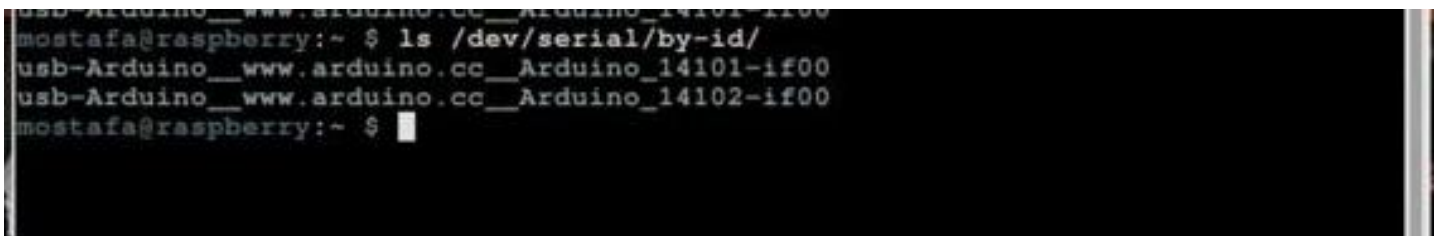


Figure 39 Detection of serial interfaces on Raspberry Pi.

Mega–Raspberry Command Protocol:

A command-based protocol was implemented between Arduino Mega and Raspberry Pi to coordinate QR recognition, robotic arm operation, and autonomous execution.

Command	Description
V	Request QR code scanning
W	Request box pickup
A	Start autonomous mission
D	Task completed successfully
N	Task execution failed

The protocol uses acknowledgement messages to synchronize system operation. The Mega controller does not proceed to the next task until a completion acknowledgement (D) is received from the Raspberry Pi.

Telemetry Communication :

Telemetry packets are periodically transmitted from Arduino Mega to Raspberry Pi to

Field	Description
121	Back-left wheel RPM
119	Back-right wheel RPM
114	Robot X position
140	Robot Y position
178	Robot heading angle

provide real-time information about robot operation and update the dashboard interface.

Field	Description
T	Packet header
3	Mission step
120	Front-left wheel RPM
118	Front-right wheel RPM

Raspberry Pi–Uno Communication :

Raspberry Pi communicates with Arduino Uno to execute robotic arm operations. The Raspberry Pi sends command sequences according to the detected QR code and the current mission requirements.

Command	Function
W	Enable pick mode
Red	Execute Red pick sequence
Green	Execute Green pick sequence
Blue	Execute Blue pick sequence
AR	Place Red box
AG	Place Green box
AB	Place Blue box

Communication Sequence During Operation :

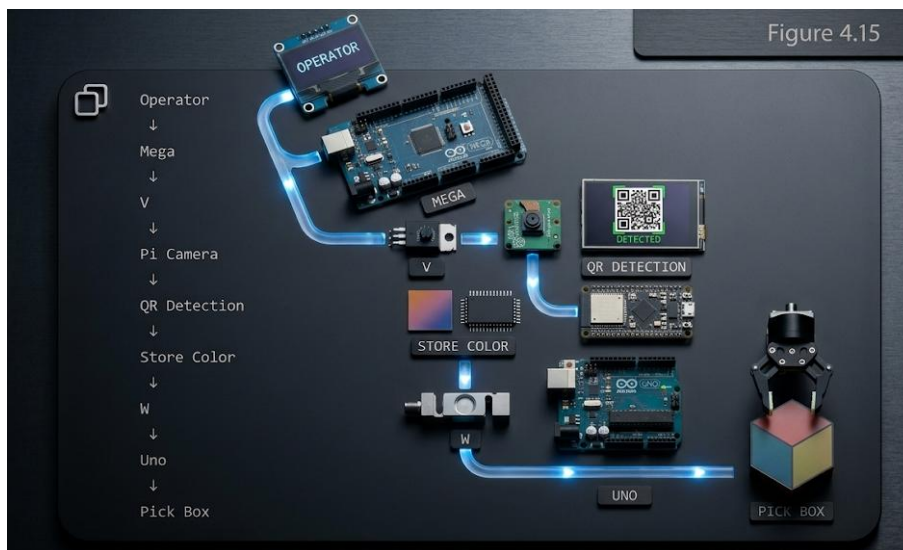


Figure 40 : Communication sequence during manual collection mode.

During autonomous mode, Arduino Mega controls robot navigation while Raspberry Pi coordinates box placement operations through Arduino Uno.

7)operating sequence

7.1) manual operating mode

The system supports a manual pick-and-place mode through Bluetooth communication. In this mode, the operator sends commands from a mobile device to the Arduino Mega. When the operator sends the command “V”, the Arduino Mega forwards the command to the Raspberry Pi. The Raspberry Pi then activates the camera and starts the object detection process.

After the camera is activated, the operator sends the command “W” to begin the picking process. The Raspberry Pi captures and processes the image, reads the QR code attached to the detected box, and determines its color classification (Red, Green, or Blue).

Based on the detected QR code, the Raspberry Pi sends the corresponding command to the Arduino Uno. The Arduino Uno then executes the appropriate pick sequence for the detected box color. After the box is successfully picked and the robot arm returns to its home position, the Arduino Uno sends the confirmation message “D” through serial communication to indicate successful completion of the operation.

7.2)autonomous operating mode

In autonomous mode, the robot follows a predefined navigation sequence controlled by the Arduino Mega. The sequence is as follows:

- Step 1: Move Forward 63 cm
- Step 2: Rotate 90 degrees CCW
- Step 3: Move Forward 118 cm
- Step 4: Place Red Box
- Step 5: Move Backward 22 cm
- Step 6: Rotate 185 degrees CCW
- Step 7: Move Left 210 cm
- Step 8: Place Green Box
- Step 9: Move Right 50 cm
- Step 10: Move Forward 145 cm
- Step 11: Diagonal Forward-Right 110 cm
- Step 12: Place Blue Box & Finish

Note : these dimensions calculated after considering slipping factor effect in different

movements

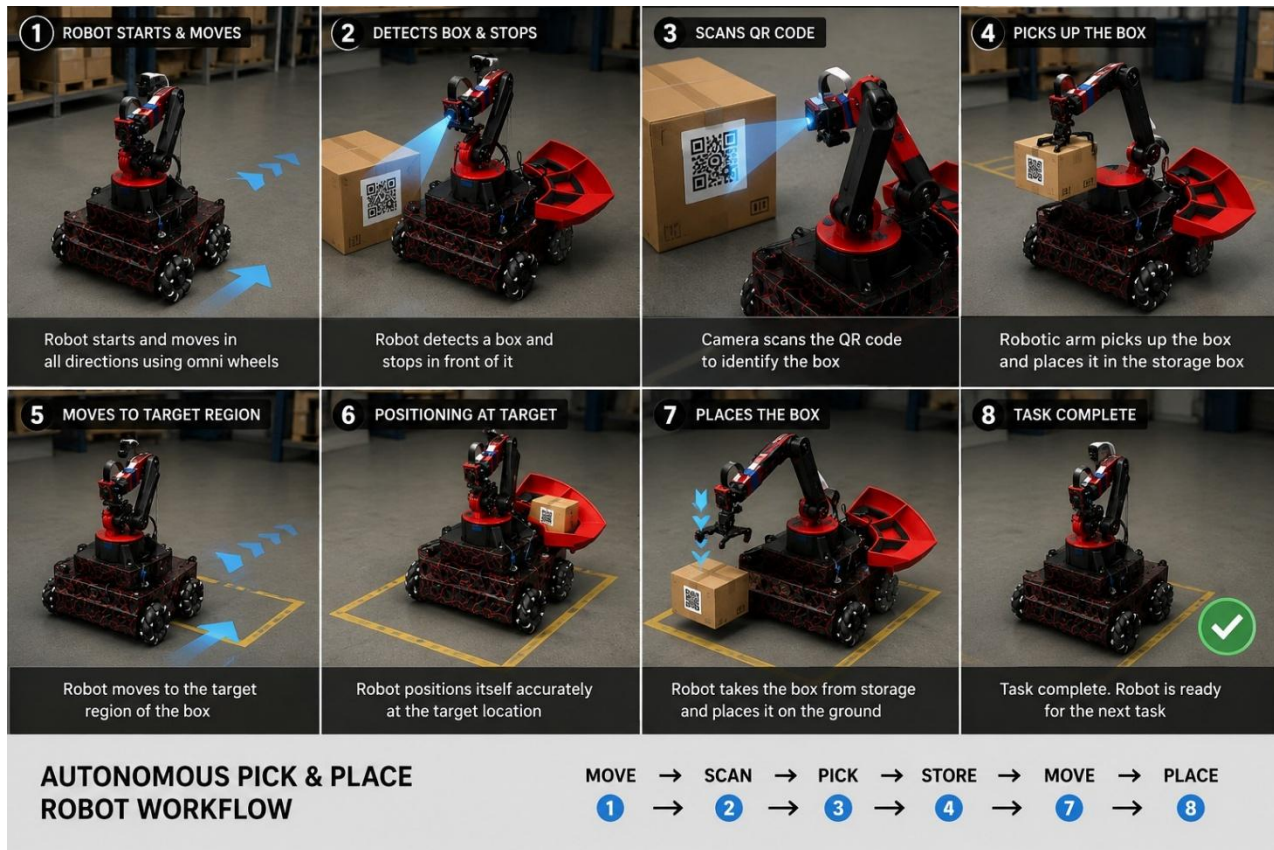


Figure 41 autonomous mode workflow

8)HMI

The dashboard was implemented using Python and the PyQt5 framework. PyQt5 was selected because it provides a powerful environment for developing interactive graphical interfaces and supports real-time updates from external data sources.

Software Tool	Function
Python	Dashboard Programming
PyQt5	Graphical User Interface
OpenCV	Image Processing
pySerial	Serial Communication

Dashboard Architecture :

The dashboard continuously receives telemetry packets from Arduino Mega through Raspberry Pi serial communication. The received data are parsed and stored in software variables, then the corresponding dashboard components are updated in real time.

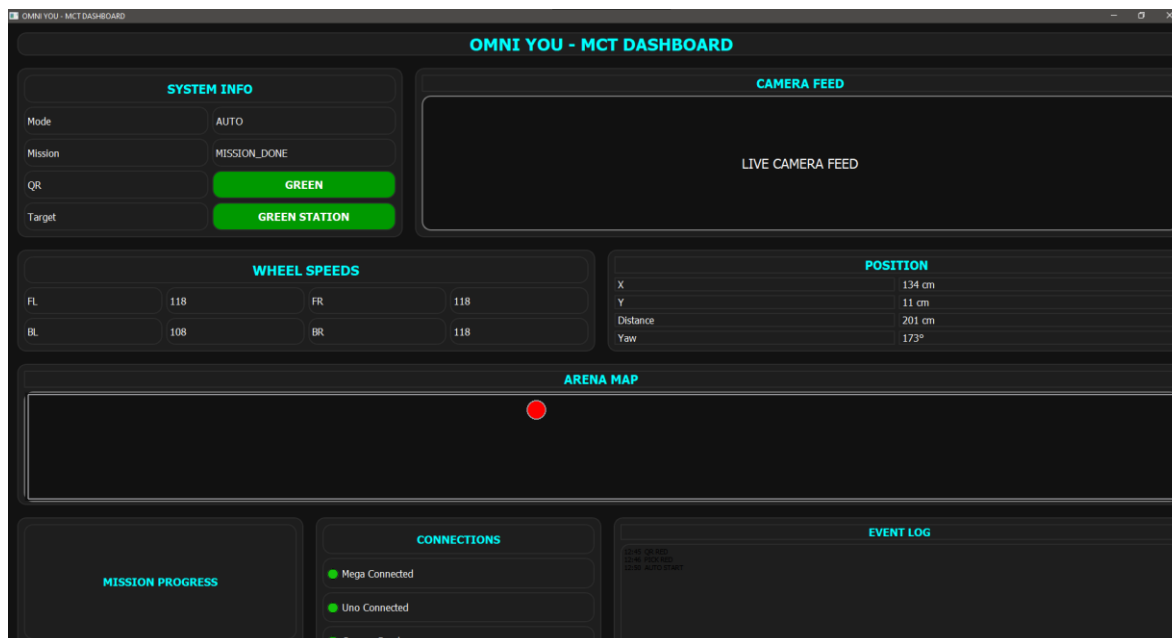


Figure 42 dashboard of HMI

Dashboard Components :

A) System Information Panel

The System Information panel displays the current operating mode, mission state, detected QR code, and destination target. This information allows the operator to continuously monitor the robot status.

B) Camera Feed Panel

The camera panel displays periodically captured images from the Raspberry Pi Camera. It provides visual feedback during QR code detection and monitoring operations

C) Wheel Speed Monitoring

The dashboard displays the rotational speed of the four mecanum wheels in revolutions per minute (RPM). Real-time speed monitoring assists in evaluating robot motion performance and detecting abnormal wheel behavior.

D) Position Monitoring

Position information including X position, Y position, travelled distance, and heading angle is displayed continuously. These parameters are estimated using encoder measurements and IMU data.

E) Arena Visualization

A simplified arena map was developed to visualize the estimated robot position in real time. The robot marker changes continuously according to the position data received from Arduino Mega.

F) Mission Progress

The mission progress bar provides a visual indication of the completion percentage of the autonomous delivery sequence.

G) Connection Indicators

Connection indicators display the communication status of Arduino Mega, Arduino Uno, and Raspberry Pi Camera, enabling quick diagnosis of communication failures

H) Event Log

The event log records important system events such as QR detection, pick operations, and



autonomous mission execution. This feature simplifies system monitoring and debugging.

Advantages of the Proposed HMI

- Real-time monitoring.
- Improved user interaction.
- Easy debugging and diagnostics.
- Visualization of navigation information.
- Simplified monitoring of autonomous operation



9) drive link

<https://drive.google.com/drive/folders/1qhX5NFdfHtmUCx4y0MpPgR20pd4Hs9K9?usp=sharing>